

STAT GR5398 – Assignment 2 Report

This report documents my implementation and empirical study for Assignment 2 of STAT GR5398 (AI & Foundations of Trading). I fine-tuned two small language models on the FinGPT Dow Jones 30 forecasting dataset using parameter-efficient LoRA adapters, and then compared their forecasting-style generation quality and inference latency against their respective base models.

1. Hardware and Environment

GPU: Google Colab Pro / Pro+ with NVIDIA A100 40GB for the final training and evaluation runs; local debugging was also done on an RTX 3060 laptop GPU. ■ System: Colab Linux runtime (CUDA toolchain pre-installed by Colab), Python 3.12. ■ Key libraries: transformers 4.46.1, peft 0.13.x, accelerate 0.34.x, datasets 2.21.0, bitsandbytes 0.44.x, rouge-score, wandb for experiment logging.

2. Dataset: FinGPT Dow30 Forecaster

I used the public FinGPT dataset FinGPT/fingpt-forecaster-dow30-202305-202405. Each example is an instruction-style prompt containing recent news and basic financials for one Dow Jones 30 company, plus a concise reference answer describing expected stock movement in the next week. I followed the provided train/validation/test split and only used the official test set (300 examples) for evaluation and comparison. During evaluation I additionally rewrote each prompt to move the guidance section right after the system intro message, following the instructor's comparison script.

3. Models and Fine-tuning Setup

Base models: • Llama3-1B: meta-llama/Llama-3.2-1B-Instruct. • DeepSeek-1.5B: deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B. ■■ I used 4-bit NF4 quantization via bitsandbytes so that both models could be trained on a single GPU. On top of the frozen base, I attached a LoRA adapter (rank $r=16$, $\alpha=32$, dropout 0.05) applied to the key, value and projection matrices of the attention and MLP blocks. I fine-tuned only the LoRA parameters and left the base weights frozen. ■■ Hyperparameters for both models were kept the same to enable a fair comparison: ■ • Epochs: 1 over the training split (full epoch). • Batch size per device: 1, with `gradient_accumulation_steps = 16` (effective batch size 16 sequences). • Max sequence length: 2,048 tokens; max generation length: 512 new tokens. • Optimizer: AdamW (via transformers Trainer) with learning rate $5e-5$. • lr scheduler: cosine with warmup ratio 0.03. • LoRA adapters and tokenizers were saved under `/content/drive/MyDrive/STAT5398_A2/outputs`.

4. Evaluation Protocol

Following the assignment instructions, I compared both base models and their LoRA fine-tuned versions on the test split using: • Rouge-1 F1 and Rouge-L F1 between generated summary and reference answer. • Exact match rate (after stripping whitespace). • Inference time per sample (seconds), measured separately for base and fine-tuned models using the same generation setup. ■■ Generation used `do_sample=True` with temperature 0.7 and `top_p=0.9` to reflect the qualitative behavior of the forecasting agent. For post-processing, I extracted the text after the `[/INST]` marker to remove the prompt from the decoded sequence.

5. Quantitative Results

The following table summarizes the Rouge-based accuracy metrics on the 300-example test

set. All values are F1 scores between 0 and 1 (higher is better).

Model	ROUGE-1 F	ROUGE-L F	Exact Match
Llama3-1B base	0.3704	0.1768	0.000
Llama3-1B fine-tuned	0.3462	0.1645	0.000
DeepSeek-1.5B base	0.3786	0.1627	0.000
DeepSeek-1.5B fine-tuned	0.3808	0.1680	0.000

Average latency per sample (seconds) across the same 300 test prompts was: • Llama3-1B base: 18.35 s • Llama3-1B fine-tuned: 28.36 s • DeepSeek-1.5B base: 29.66 s • DeepSeek-1.5B fine-tuned: 41.57 s ■ Exact match is 0 for all models, which is expected because the reference answers are short, discrete labels while the models generate long natural-language analyses. Rouge therefore better captures task performance in this setting.

6. Discussion

1) Accuracy vs. model family: ■ DeepSeek-1.5B consistently achieves slightly higher Rouge-1 and Rouge-L scores than Llama3-1B, for both base and fine-tuned variants. This suggests that the DeepSeek family, even at 1.5B parameters, is somewhat better aligned with financial reasoning and summarization tasks. ■ ■ 2) Effect of LoRA fine-tuning: ■ For Llama3-1B, fine-tuning slightly decreases Rouge scores. Qualitatively, the fine-tuned model tends to generate longer, more narrative explanations, which can reduce n-gram overlap with the concise ground-truth labels. For DeepSeek-1.5B, LoRA fine-tuning brings a small but consistent improvement in both Rouge-1 and Rouge-L. The gains are modest but show that the adapter is successfully specializing the model towards the Dow30 forecasting style. ■ ■ 3) Latency trade-offs: ■ Within each family, the fine-tuned models are noticeably slower than the base models. This is partly due to the extra LoRA layers and partly due to slightly longer outputs. Across families, DeepSeek-1.5B (base and fine-tuned) is significantly slower than Llama3-1B, which is expected from the larger parameter count and more complex decoder architecture. In a real-time trading assistant, Llama3-1B might be preferable if hard latency constraints dominate, while DeepSeek-1.5B offers a better accuracy–speed trade-off when a few extra seconds are acceptable.

7. Qualitative Observations

Qualitative inspection of randomly selected test prompts shows the following patterns: • Llama3-1B base often produces short, to-the-point answers that roughly match the direction ("up" vs. "down") but lack detailed justification. • Llama3-1B fine-tuned tends to enumerate both positive and negative factors more explicitly, but sometimes hedges instead of giving a clear direction. ■ • DeepSeek-1.5B base produces more structured paragraphs with explicit bullet-like reasoning (risk vs. opportunity), which aligns well with the desired style. • DeepSeek-1.5B fine-tuned best mimics the instructor-provided answers, explicitly stating the expected stock movement and giving 2–4 key drivers, albeit with some hallucinated details typical of large language models.

8. Conclusion

In this assignment I successfully fine-tuned two small language models on a Dow30 forecasting dataset using 4-bit LoRA, and compared their generation quality and latency against their base versions. DeepSeek-1.5B (both base and fine-tuned) achieves slightly higher Rouge scores

than Llama3-1B, while the fine-tuned DeepSeek adapter provides the best overall accuracy at the cost of substantially higher inference time.■■■The results highlight an important trade-off for practical financial applications: larger or more heavily fine-tuned models offer more detailed and accurate reasoning, but they incur significant latency and computational cost. A promising direction for future work would be to distill the fine-tuned DeepSeek model into a smaller student model, or to use retrieval-augmented generation so that the LLM focuses on reasoning over carefully selected, low-entropy inputs.